Student notes — Session 5 · Loops, while and for

Slides: [`Class_05_Loops_while_and_for.pdf`](../slides/Class_05_Loops_while_and_for.pdf). These notes are the notes pane of the student copy of the deck.

Slide 1 — Loops in C: while and for

What this week is about. Last week your program learned to **choose**: `if` this, `else` that. But it still ran from the top to the bottom once, and ended. This week your program learns to go **back** and do something **again**. That is called a **loop**.

The three lines on the slide are a loop:

```
for (int i = 1; i <= 5; i++) {
    printf("%d\n", i);
}
```

The screen shows:

```
1
2
3
4
5
```

One `printf`, five lines on the screen. By slide 70 you will understand every part of the first line.

Your Quiz 1 paper came back in this class. The mark is also in Blackboard: open the course, click **Gradebook**, and look for **Quiz 1**. Read the comments on your paper. If you think the points were added up wrongly, bring the paper to the instructor after a class, or book office hours.

A vote is open on Blackboard about the number of quizzes: 6 quizzes with the best 5 counting (as now), or 8 quizzes with the best 6 counting. The Blackboard announcement has the details. Please vote.

The programs of this class are on Blackboard. Open the course, open the **Week 3** module, and download the item **Class 5 code**. It is a zip file with 20 small programs, the same ones you see on the slides. Each file starts with a comment that says what the program does, how to compile it and how to run it. Under that is a list called **Try**: three or four small changes to make. Do them. Changing a program and watching what happens is the best way to learn it.

How to try an example. Same as before:

1. Open Visual Studio Code. Click **File**, then **Open Folder**, and open a new folder called `c_week3`.
2. Click **File**, then **New Text File**. Type the program. Click **File**, then **Save As**, and give it a name ending in `.c`, for example `test.c`.
3. Click **View**, then **Terminal**. In the terminal type `gcc -Wall test.c -o test` and press Enter. If nothing is printed, it worked.
4. Type `./test` and press Enter. (In Windows PowerShell, type `.\test` instead.)

Many examples in these notes are only a few lines. To run them, put the lines inside this frame, where the `...` is:

```
#include <stdio.h>
int main(void) {
    ...
```

```
    return 0;
}
```

If a program does not stop: click inside the terminal, hold the **Ctrl** key and press **C**. More about this on slide 58.

Slide 2 — Quiz 1 is back

Slides 2 to 8 are the three mistakes that cost the most marks on Quiz 1, across the whole class. None of them is difficult. All three are **habits** — and a habit is cheap to change once you have seen it.

Read these seven slides again before the next quiz. They are the cheapest marks you will ever get back.

Slide 3 — Mistake 1: `%f` always prints six decimals

This was question 1(b), and it cost marks on more papers than anything else in the quiz.

`x` is a `double` holding `6.25`. Three `printf` lines give three different answers:

```
double x = 6.25;
printf("%.2f\n", x);
printf("%f\n", x);
printf("%.0f\n", x);
```

The screen shows:

```
6.25
6.250000
6
```

- `%.2f` — **you** choose how many decimals. Here, two.
- `%f` — you choose nothing, and it always prints **six** decimals. `6.250000`. Every time. That is simply what `%f` means.
- `%.0f` — no decimals at all.

The rule to remember: if you want two decimals, you have to write the `.2`. `%f` on its own is always six.

Slide 4 — `%.0f` rounds. It does not cut.

`%.0f` does not chop the digits off the end. It **rounds** to the nearest whole number.

x holds	%.2f	%f	%.0f
6.25	6.25	6.250000	6
2.75	2.75	2.750000	3

`6.25` rounds **down** to `6`. But `2.75` rounds **up** to `3`, not down to `2`. If you had variant B of the quiz and wrote `2`, this is the line you lost.

Slide 5 — Mistake 2: `scanf` needs `%lf` and `&`

This one cost whole points, not halves. It was the first bug in question 1(c), and it was the `scanf` line in question 2.

There are two separate things to get right, and they are both on the same line.

One: the hole is different. For a `double`, `printf` takes `%f`, but `scanf` takes **`%lf`** — with the letter `l`, for *long*. They are not the same and you cannot swap them.

Two: `scanf` needs the `&`. `scanf` has to be told **where to put** the number it reads. The `&` in front of the variable is what tells it. Leave the `&` out and `scanf` does not know where your box is.

What many people wrote:

```
double price;
scanf("%f", &price);

double kg;
scanf("%lf", kg);
```

What it has to be:

```
double price;
scanf("%lf", &price);

double kg;
scanf("%lf", &kg);
```

Learn the whole line as one piece: `scanf("%lf", &x);`

Slide 6 — The compiler did tell you

You were not on your own with that mistake. The compiler saw it.

Compiled with `gcc -Wall`, the wrong version says:

```
warning: format '%f' expects argument of type 'float *',
but argument 2 has type 'double *'
```

It names the line and it names the problem.

And here is the part that catches people out: **it still builds a program**. The program runs. It prints something. When this one was run it printed `Total: 0.00` — a perfectly calm, completely wrong answer. On another computer it might print a different wrong number, or crash.

So: compile with `-Wall`, and **read what it says** before you run. A warning is not noise. This mistake was free to find.

Slide 7 — Mistake 3: the small things that stop a program

The third one is not a single mistake, it is a family of small ones. Each costs half a point:

1. Every statement ends with a **semicolon** `;`
2. `main` ends with **`return 0;`** and a closing brace `}` — every time, the same shape.
3. It is `#include <stdio.h>`. Not `studio.h`. Not `# insert`.
4. C is **case sensitive**: `printf`, never `Printf`.

The good news: the compiler finds every single one of these for you, and tells you the line. Compile early and compile often — after every few lines, not at the end — and you will never lose these marks again.

Slide 8 — All three come back today

None of this is behind you. Look at what this class is about: every loop you write reads a number, counts, and prints a result. The same `scanf`. The same `%.2f`. The same semicolon.

That is what a quiz is for. It is small, and its job is to tell you **early** what you need to practise.

The practice test on Blackboard asks the same kind of question, as often as you like, and it is not graded. If any of these three slides was news to you, that is where to go next.

Slide 9 — Today

Eight steps in this class:

1. Why loops exist.
2. The four parts of a loop.
3. The `while` loop.
4. Tracing a loop with a pen.
5. The loop that never stops.
6. The `for` loop.
7. `++` and `--`: add one, take one away.
8. `+=`, `-=`, `*=`, `/=`, `%=`: short ways to change a variable.

The goal is to **repeat tasks safely**, not to memorise loop patterns. "Safely" means two things. You know where your loop starts. And you know it will stop.

The next class uses loops for real work: adding up numbers, counting things, asking again when the input is wrong.

Slide 10 — Why loops exist

This is the first part of the class: why a program needs loops at all. Slides 11 to 17 belong to it.

Slide 11 — From choosing to repeating

Until now, every program you wrote ran from the first line to the last line, once. An `if` could skip some lines, but C never went **back**.

A **loop** lets the program go back up and run the same lines again. You write the lines once. The loop decides how many times they run.

Slide 12 — Print Hello five times

The problem. Print `Hello` five times.

The program on this slide does it without a loop: the same `printf` line, five times. Copy, paste, paste, paste, paste. It works. The screen shows:

```
Hello
Hello
Hello
Hello
Hello
```

This is `01_hello_copy_paste.c` in the Class 5 code.

Slide 13 — A new request

Now somebody asks for `Hello` **1000 times**, not 5.

With the program of slide 12 you must add 995 more lines. Nobody wants to do that, and nobody can check afterwards that there are exactly 1000. Think about it for a moment, then go to the next slide.

Slide 14 — The same five Hellos, with a loop

The three bright lines are a loop:

```
for (int i = 0; i < 5; i++) {
    printf("Hello\n");
}
```

There is only **one** `printf`. The line above it says: do this 5 times. The screen shows exactly the same five `Hello` lines as on slide 12.

You do not need to understand the `for` line yet. Slides 62 to 70 explain every part of it. This is `02_hello_loop.c` in the Class 5 code.

Slide 15 — From 5 to 1000

Left: copy and paste. Right: the loop.

To go from 5 to 1000, the left side needs 995 more lines. The right side needs a change of **one number**: `5` becomes `1000`. That is the first reason for loops: you write the action once, and a number controls how often it runs.

Slide 16 — When you do not know the number

Here is the second reason, and it is the stronger one. "Print Hello as many times as the user says." When you write the program, you **do not know the number yet**. You cannot copy and paste a number you do not know.

With a loop it is easy, because the number can be a variable. The program asks, `scanf` puts the answer into `n`, and the loop runs `n` times:

```
for (int i = 0; i < n; i++) {
    printf("Hello\n");
}
```

You type	The screen shows
`3`	`Hello` three times
`0`	nothing after the question
`-2`	nothing after the question

With `0` and with `-2` the loop runs zero times. That is not an error: the loop asks "is `i` smaller than `n`?", the answer is no at once, and the program goes on. Slide 43 says more about this.

This is `03_hello_n_times.c` in the Class 5 code. Run it and try `3`, `10`, `0` and `-2`.

Slide 17 — What loops are for

Four everyday jobs for a loop:

1. **Count:** 1, 2, 3, up to 100.
2. **Read many values:** 30 marks, one after another.
3. **Search:** look at every item until you find the one you want.

4. **Ask again:** until the input is right.

All four are "do the same thing again". Almost every real program has a loop in it.

Slide 18 — The four parts of a loop

This is the second part of the class, slides 19 to 25. Every loop, in every programming language, has the same four parts: **start, check, body, update**. Learn these four words. They are the most important thing in this class.

Slide 19 — The four parts: start

Start: make a variable and give it its first value.

```
int count = 1;
```

The start happens **once**, before the loop begins. It never runs again.

Slide 20 — The four parts: check

Check: a yes/no question, a condition, exactly like the one in an `if`.

```
count <= 3
```

If the answer is yes, the loop goes on. If the answer is no, the loop is over.

Slide 21 — The four parts: body

Body: the work you want repeated. These are the lines inside `{ }`, for example a `printf`. The body is the reason you wrote the loop.

Slide 22 — The four parts: update

Update: change the variable, so that one day the answer to the check is **no**.

```
count++;
```

`count++` means "add 1 to `count`". Without an update, the check gives the same answer for ever, and the loop never stops (slide 56).

Slide 23 — Start, check, body, update

All four together:

Part	What it does	Example
Start	Give a variable its first value. Once.	`int count = 1;`
Check	A yes/no question. No: the loop is over.	`count <= 3`
Body	The work you want repeated.	`printf(...);`
Update	Change the variable.	`count++;`

The order. Start. Then: check, body, update. Check, body, update. And so on, until the check says no. After the update C goes back to the **check**, not to the start.

Slide 24 — One trip around is one round

Start, then check, body, update, check, body, update, check, and so on.

One trip through check, body, update is called a **round**. Books also call it an **iteration**. When these notes say "the loop ran 4 rounds", the body ran 4 times.

Slide 25 — When a loop never stops

The question on the slide: a loop never stops; which of the four parts is probably wrong? The answer: the **update**. It is missing, or it goes the wrong way. Sometimes it is the check.

When a loop does something wrong, one of the four parts is wrong:

What you see	Which part to look at
The loop never stops	The update is missing, or it goes the wrong way.
One round too many, or one too few	The check: `<` or `<=`?
The loop starts from the wrong number	The start.
The loop does nothing at all	The check is false at the very beginning.

The habit: before you run a loop, point at each of the four parts in your code and say its name.

Slide 26 — The while loop

The third part of the class, slides 27 to 44: the first kind of loop in C, the `while` loop. In one sentence: a `while` is an `if` that keeps asking.

Slide 27 — while looks like if

The shape.

```
while (condition) {
    // repeated lines
}
```

It looks like an `if`: a word, a condition in round brackets, a block in curly brackets. There is one difference. After an `if` block, C moves on. After a `while` block, C goes **back up** and asks the condition again. **A `while` is an `if` that keeps asking.**

Slide 28 — How a while loop runs

Four steps:

1. Check the condition.
2. If it is true, run the body.
3. Go back to the condition. Check again.
4. If it is false, leave the loop. C goes on with the line after the closing `}`.

Slide 29 — Type along: count.c

This is the first loop typed in class, the whole program. It is `04_count_while.c` in the Class 5 code; in class the file was called `count.c`.

```
#include <stdio.h>

int main(void) {
    int count = 1;

    while (count <= 3) {
```

```
        printf("%d\n", count);
        count++;
    }

    printf("Done\n");
    return 0;
}
```

Type it yourself, do not copy it. The next slides go through it line by line.

If your program does not work, check:

1. There is **no semicolon** after `while (count <= 3)`. With a semicolon there, the program prints nothing and never ends (slide 61). Press **Ctrl+C**, delete the semicolon.
2. `count++;` is **inside** the braces.
3. You wrote `<=` and not `<`. With `<` the screen shows only `1` and `2`.

Slide 30 — The start

The bright line is the **start**: `int count = 1;`. It makes one box, called `count`, and puts 1 in it. This line runs **once**.

Slide 31 — The check

The bright line is the **check**: `while (count <= 3)`. It asks: is `count` at most 3? **Yes:** run the body. **No:** jump past the closing `}`.

Slide 32 — The body

The two bright lines are the **body**: everything inside `{ }`. They run once in every round. The first line prints what is in the box. The second line is also the update (next slide).

Slide 33 — The update

The bright line is the **update**: `count++;`. It means "add 1 to `count`". It is short for `count = count + 1`. Without this line the loop never stops, because `count` stays 1 and `1 <= 3` is always true.

Slide 34 — The closing brace

When C reaches the closing `}` of the loop, it goes **back up** to the check, the `while` line. It does **not** go back to `int count = 1;`. If it did, `count` would be 1 again every time and the loop would never end.

Slide 35 — After the loop

`printf("Done\n");` is **outside** the loop, after the closing `}`. So it runs once, when the loop is over. The screen shows:

```
1
2
3
Done
```

Slide 36 — Compile it and run it

In the terminal:

```
gcc -Wall count.c -o count
./count
```

In Windows PowerShell the second line is `.\count`. If the first line prints nothing, the program compiled without problems.

Try it yourself. Change the program so that it counts from 1 to 10. (You change one number.) Then change it so that it counts from 5 to 8. (You change two numbers. Expect `5`, `6`, `7`, `8`, `Done`.)

Slide 37 — Walk through it: check 1

Slides 37 to 40 go through `count.c` the way C does it, one check at a time. The table on the right gets one new row on each slide.

Check 1: `count` is 1. Is `1 <= 3`? Yes. Print `1`. `count` becomes 2. Go back up.

Slide 38 — Walk through it: check 2

Check 2: `count` is 2. Is `2 <= 3`? Yes. Print `2`. `count` becomes 3. Go back up.

Slide 39 — Walk through it: check 3

Check 3: `count` is 3. Is `3 <= 3`? Yes, because `<=` means "smaller **or equal**". Print `3`. `count` becomes 4. Go back up.

Slide 40 — Walk through it: check 4

Check 4: `count` is 4. Is `4 <= 3`? **No.** C leaves the loop and runs the line after it: print `Done`.

Count the rows: the check ran 4 times, the body ran 3 times. The last check is the one that says no.

Slide 41 — Change one thing

The same loop, but the start is now `int count = 10;`. Before you go to the next slide, decide: what is printed? This is `05_count_zero_times.c` in the Class 5 code.

Slide 42 — Change one thing: the answer

The screen shows only:

```
Done
```

Is `10 <= 3`? No, at the very first check. So the body never runs, not even once. C jumps past the `}` and prints `Done`.

Slide 43 — Zero rounds is normal

A `while` loop checks before every round, also before the first one. If the check is false at the start, the body runs **zero times**. That is not an error. The program does not stop; it just goes on with the line after the loop.

You saw the same thing on slide 16: when the user typed `0`, no `Hello` was printed.

Slide 44 — One more: counting down

A loop does not have to count up. This one starts at 5, goes on while `n > 0`, and takes 1 away in every round with `n--`. The screen shows:

```
5
4
3
2
1
Liftoff!
```

Find the four parts. Start: `int n = 5;`. Check: `n > 0`. Body: the `printf`. Update: `n--;`. `Liftoff!` is after the loop, so it is printed once. When the loop ends, `n` is 0: that is the first value for which `n > 0` is false.

This is `09_countdown_while.c` in the Class 5 code. Try: start at 10; change `> 0` to `>= 0` and see that the last number is now `0`.

Slide 45 — Tracing a loop with a pen

The fourth part of the class, slides 46 to 54. **Tracing** means: you play the computer, with a pen. **Do not guess the output. Track the values.**

Slide 46 — Why trace?

On a quiz you have no compiler. You have a pen. Tracing is how you find the output.

Draw a table. **One row per round.** In each row write what the variable holds, the answer to the check, and what is printed.

Slide 47 — Trace it on paper

Try it before you look at the next slides. Take pen and paper.

```
int x = 1;

while (x <= 4) {
    printf("%d ", x);
    x = x + 1;
}
```

Two questions: what does it print, and what is `x` **after** the loop? (`x = x + 1` does the same as `x++`.) This is `06_trace_while.c` in the Class 5 code.

Slide 48 — The trace table: row 1

Slides 48 to 52 fill in the table one row at a time.

Round 1: `x` is 1. Is `1 <= 4`? True. Print `1`. Then `x` becomes 2.

Slide 49 — The trace table: row 2

Round 2: `x` is 2. True. Print `2`. Then `x` becomes 3.

Slide 50 — The trace table: row 3

Round 3: `x` is 3. True. Print `3`. Then `x` becomes 4.

Slide 51 — The trace table: row 4

Round 4: `x` is 4. Is `4 <= 4`? True, because of the `=` in `<=`: 4 is equal to 4. Print `4`. Then `x` becomes 5.

Slide 52 — The trace table: row 5

The last row: `x` is 5. Is `5 <= 4`? **False.** The loop ends. Nothing is printed in this row. The complete table:

Round	x	x <= 4	prints	then x
1	1	true	1	2
2	2	true	2	3
3	3	true	3	4
4	4	true	4	5
	5	false		

Slide 53 — What the table tells you

- The screen shows `1 2 3 4`.
- After the loop, `x` is **5**, not 4.
- The body ran **4** times.
- The check ran **5** times: four times true, one time false. The check always runs one more time than the body.

Slide 54 — A favourite quiz question

"What is `x` after the loop?" It is **not** the last number printed. It is the **first number that failed the check**.

Why 5? The loop only ends when the condition is false. For `x <= 4` to be false, `x` had to become 5.

You can see it yourself. `06_trace_while.c` has this line after the loop:

```
printf("\nAfter the loop x is %d\n", x);
```

The screen shows:

```
1 2 3 4
After the loop x is 5
```

Try it yourself. Trace this on paper first, then run it:

```
int x = 3;
while (x <= 9) {
    printf("%d ", x);
    x = x + 3;
}
printf("\nx is %d\n", x);
```

(Answer: the screen shows `3 6 9` and then `x is 12`. The body runs 3 times.)

Slide 55 — The loop that never stops

The fifth part of the class, slides 56 to 61. You will write a loop that never stops, by accident. Everyone does. After these slides you know what it looks like and how to get out.

Slide 56 — The update is missing

A loop must, one day, make its condition false. If it never does, the loop never ends. That is called an **infinite loop**.

On the slide the update is gone; there is only a comment in its place. `count` is 1. Is `1 <= 3`? Yes. Print `1`. Go back up. `count` is still 1. Yes. Print `1`. Still 1. For ever. The screen fills with `1`, line after line, very fast, and `Done` is never printed.

This is `07_infinite_loop.c` in the Class 5 code. Read slide 58 before you run it.

Slide 57 — The compiler says nothing

This program compiles with `gcc -Wall` and **no message at all**. A loop that never ends is perfectly spelled C. The compiler checks your **spelling**, not your **thinking**. So "it compiled without warnings" does not mean "it is right".

Slide 58 — How to stop a running program: Ctrl + C

1. Click once inside the terminal.
2. Hold the **Ctrl** key.
3. Press **C**. Once.

The program stops and the terminal is ready for your next command. This is written **Ctrl+C**. It works on Windows, macOS and Linux. On a Mac it is the **Control** key too, not Command.

If nothing happens, click inside the terminal and press Ctrl+C again. If that still does not work, click the small **dustbin icon** at the top right of the terminal panel in VS Code, then click **View**, then **Terminal** to open a new one.

Do not close VS Code and do not switch off the laptop. Ctrl+C is enough.

Slide 59 — Your turn: break it, stop it, fix it

Do this once, calmly, so that it will not frighten you later:

1. Open your `count.c`. Put `//` in front of `count++;`. The line is now a comment.
2. Compile with `-Wall`. There is no warning.
3. Run it. A wall of 1s.
4. Stop it with **Ctrl+C**.
5. Take the `//` away again. Compile. Run. The screen shows `1`, `2`, `3`, `Done`.

Slide 60 — Never stops / stops

Left: the loop without an update. Right: the same loop with `count++;` back in its place. One line is the whole difference.

The habit. Before you run a loop, put your finger on the update: the line that changes the variable in the condition. If you cannot find one, do not run it.

Slide 61 — A silent cousin: the semicolon

Look at the end of the bright line: `while (count <= 3);`. That semicolon is an **empty body**. The loop is `while (count <= 3);` and nothing else. The block in `{ }` under it is no longer part of the loop.

`count` never changes, so the loop never ends, and it prints **nothing**. The program just seems frozen.

This time the compiler helps. With `-Wall` it says:

```
warning: this 'while' clause does not guard...
```

If your program seems frozen: press Ctrl+C, then look for a semicolon straight after `while (...)`, and delete it.

This is `08_while_semicolon_bug.c` in the Class 5 code. Compile it and read the warning. If you run it, stop it with Ctrl+C.

Slide 62 — The for loop

The sixth part of the class, slides 63 to 76: the second kind of loop. A `for` loop puts the start, the check and the update on **one line**.

Slide 63 — Three parts in three places

Look at the `while` loop again. The start is on a line **above** the loop. The check is in the brackets. The update is at the **bottom** of the body, where it is easy to forget, as you saw on slide 56.

Counting loops are so common that C has a second shape for them.

Slide 64 — The shape of a for loop

```
for (start; check; update) {
    body
}
```

Books call the three parts initialization, condition and update. Same three things.

Two spelling rules.

1. Between the three parts there are **semicolons** `;`, not commas.
2. After the closing round bracket there is **no semicolon**. `for (...) ;` is an empty loop, the same trap as on slide 61.

Slide 65 — When each part runs

For the line `for (int i = 1; i <= 5; i++)`:

Part	When it runs
start, `int i = 1`	once, at the beginning
check, `i <= 5`	before each round. Yes: run the body. No: the loop is over.
update, `i++`	after each round, after the body

Read the line aloud: start `i` at 1; go on while `i` is at most 5; after each round, add 1 to `i`.

Why the name `i`? Tradition. It is short for "index". You may use any name, for example `count`.

Slide 66 — Type along: for.c

The second program typed in class. It is `10_for_count.c` in the Class 5 code; in class the file was called `for.c`.

```
#include <stdio.h>

int main(void) {
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
```

```
    printf("\n");
    return 0;
}
```

The screen shows:

```
1 2 3 4 5
```

The `printf("\n");` after the loop only moves to a new line, so that the terminal prompt does not stick to the 5.

Slide 67 — The order inside a for: round 1

Slides 67 to 69 show the order with a table, like the one on slides 48 to 52.

First row: the start makes `i` and puts 1 in it. Check: `1 <= 5` is true. Body: print `1`. Update: `i` becomes 2.

Slide 68 — The order inside a for: round 2

Second row: check again. `i` is 2, `2 <= 5` is true. Print `2`. `i` becomes 3. The start does **not** run again.

Slide 69 — The order inside a for: the whole table

The whole table. The rounds go on until `i` is 6. Is `6 <= 5`? False. The loop ends. The body ran 5 times, the check ran 6 times: the same rule as on slide 53.

So the order is: start, check, body, update, check, body, update, and so on. That is why the first number printed is 1 and not 2: the update runs **after** the body.

Slide 70 — The title slide, answered

The three lines on slide 1 print 1, 2, 3, 4, 5. Now you can read every part of the first line.

The question on the slide: which part changes `i` after each round? The third one, `i++`.

Slide 71 — i lives only inside the loop

The bright line tries to use `i` **after** the loop. This does not compile:

```
error: 'i' undeclared (first use in this function)
```

The box `i` was made on the `for` line, so it exists only inside the loop. When the loop ends, the box is gone.

This is `11_for_scope_error.c` in the Class 5 code. It does not compile on purpose. Its "Try" list shows two ways to fix it.

Slide 72 — Make it count to 10

Change `for.c` so that it prints 1 to 10. How many characters did you change?

(Answer: change `i <= 5` to `i <= 10`. One number. The screen shows `1 2 3 4 5 6 7 8 9 10`. This is slide 15 again, in your own hands.)

Slide 73 — The body can use the counter

The body can do more than print `i`. It can **calculate** with it. The bright line prints `i` and `i * i`:

```
1 squared is 1
```

```
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25
```

In round 1, `i` is 1, so `i * i` is 1. In round 4, `i` is 4, so `i * i` is 16. Same line of code, a different value of `i` in every round.

This is `12_for_squares.c` in the Class 5 code. Try: print the squares up to 10; print `7 * i` instead, and you have the 7 times table.

Slide 74 — The same loop, two ways

Left, a `while`. Right, a `for`. Both show `1 2 3 4 5` on the screen.

Find the four parts in each one.

Part	In the `while`	In the `for`
Start	`int i = 1;` above the loop	first thing in the brackets
Check	`i <= 5` in the brackets	second thing in the brackets
Body	inside `{ }`	inside `{ }`
Update	`i++;` at the bottom of the body	third thing in the brackets

Same four parts, in different places. Anything you can write with one, you can write with the other.

One small difference. In the `while` version, `i` was made **above** the loop, so it still exists afterwards, and it holds 6. In the `for` version, `i` is gone after the loop (slide 71).

`13_same_loop_two_ways.c` in the Class 5 code has both loops in one program.

Slide 75 — Which one do I use?

- **`for`**: you **know** how many rounds. "5 times", "`n` times", "from 1 to 100".
- **`while`**: you **do not know** how many rounds. "Keep going until something happens."

Slide 76 — for or while?

Task	Which loop	Why
Print the names of the 12 months	`for`	You know it is 12.
Keep asking for the password until it is right	`while`	It may take 1 try or 20. You do not know.
Add up the numbers from 1 to 100	`for`	You know the range.

Try it yourself. Here is a `for` loop. Write it again as a `while` loop. Both must show `10 20 30 40 50`.

```
for (int i = 10; i <= 50; i += 10) {
    printf("%d ", i);
}
```

(`i += 10` means "add 10 to `i`". Slide 90 explains it. In your `while` version the start goes above the loop and the update goes at the bottom of the body.)

Slide 77 — ++ and --

The seventh part of the class, slides 78 to 88. `++` adds one. `--` takes one away. You have used `count++` all through this class; now the details.

Slide 78 — Short forms for "change by one"

You write	It means	Say it as
`i++;` or `++i;`	`i = i + 1;`	add 1 to `i`
`j--;` or `--j;`	`j = j - 1;`	take 1 from `j`

On a line of their own, `i++` and `++i` do exactly the same thing. (Slides 83 to 87 show the one case where they differ.)

```
int a = 5;
a++;
++a;
printf("%d\n", a);
```

The screen shows `7`. Each of the two lines added 1.

Slide 79 — Counting down with for

```
for (int i = 5; i >= 1; i--) {
    printf("%d ", i);
}
```

The screen shows `5 4 3 2 1`. Start at 5; go on while `i` is at least 1; after each round, take 1 away. This is `14_countdown_for.c` in the Class 5 code.

Slide 80 — Up and down: all three parts change

	Counting up	Counting down
Start	the small number: `i = 1`	the big number: `i = 5`
Check	`i <= 5`	`i >= 1`
Update	`i++`	`i--`

All **three** parts change. If you change only one or two of them, you have a bug.

Slide 81 — A habit gone wrong

You change the start and the check for counting down, and by habit you still type `i++`. Before you look at the next slide: what happens? Which of the four parts is wrong?

Slide 82 — A habit gone wrong: the answer

The screen shows `5 6 7 8 9 10 11 ...` and never stops. `i` gets bigger and bigger, so `i >= 1` is always true. The **update** goes the wrong way. Stop it with Ctrl+C (slide 58), then change `i++` to `i--`.

This is `15_countdown_wrong_update.c` in the Class 5 code. It does not stop by itself.

Slide 83 — a++ and ++a

`a++` and `++a` both add 1 to `a`. On a line of their own they are the same. The difference shows **only when you use the value in the same line**, for example `int b = a++;`. The next two slides show both cases.

Slide 84 — a++: use, then add

```
int a = 5;
int b = a++;
```

```
printf("a is %d, b is %d\n", a, b);
```

The screen shows `a is 6, b is 5`. The `++` is **after** the `a`. So C first uses `a`: `b` gets the **old** value, 5. After that, `a` becomes 6.

Slide 85 — ++a: add, then use

```
int a = 5;
int b = ++a;
printf("a is %d, b is %d\n", a, b);
```

The screen shows `a is 6, b is 6`. The `++` is **before** the `a`. So C first adds: `a` becomes 6. Then `b` gets 6.

How to remember it. Read from left to right. In `a++` you see the `a` first: the value is taken first. In `++a` you see the `++` first: the adding happens first. In both cases `a` ends up as 6. Only `b` is different.

`16_post_and_pre_increment.c` in the Class 5 code shows both cases in one program.

Slide 86 — Hands up

```
int x = 5;

printf("%d ", x++);
printf("%d\n", x);
```

What is printed: `5 6`, `6 6` or `5 5`? Decide before you go on. This is `17_predict_x_plus_plus.c` in the Class 5 code.

Slide 87 — Hands up: the answer

The screen shows `5 6`. The first `printf` gets the old value, 5. Then `x` becomes 6. The second `printf` shows 6.

With `++x` in the first `printf`, the screen shows `6 6`.

Slide 88 — The rule for this course

Write `i++` on a line of its own, or as the third part of a `for`. Do not put `++` inside a `printf` or inside a calculation. You must be able to **read** lines like `b = a++`, because they can be on a quiz. You never need to **write** them.

A line you should never write: `printf("%d", i++ + ++i);`. It changes `i` twice in one line. Even the compiler objects: with `-Wall` it says `warning: operation on 'i' may be undefined`. Different compilers may print different numbers for it. **Rule: change a variable only once in a line.**

Try it yourself. Before you run it, say what this shows:

```
int j = 3;
j--;
printf("%d\n", j);
```

(Answer: `2`. `j--` on its own line just takes 1 away.)

Slide 89 — += and its family

The last part of the class, slides 90 to 100: a shorter way to write lines like `x = x + 3`.

Slide 90 — Take, add, put back

`x = x + 3;` means: take what is in the box `x`, add 3, put the result back in the box. You write this so often that C has a short form: `x += 3;`. Say it as "add 3 to `x`". Both lines do exactly the same thing.

Slide 91 — All five

Short form	Long form	Say it as
`x += 3;`	`x = x + 3;`	add 3 to `x`
`x -= 2;`	`x = x - 2;`	take 2 from `x`
`x *= 4;`	`x = x * 4;`	multiply `x` by 4
`x /= 5;`	`x = x / 5;`	divide `x` by 5
`x %= 2;`	`x = x % 2;`	keep the remainder of `x` divided by 2

So `x++` is the same as `x += 1`, which is the same as `x = x + 1`. Three ways to write one thing.

It works for `double` too.

```
double price = 80.0;
price *= 1.25;
printf("%.2f\n", price);
```

The screen shows `100.00`. The price went up by a quarter.

Slide 92 — Row by row

`x` starts at 10. Then five lines change it, one after another. Write down what `x` holds after each line before you go to the next slide. Remember: each line uses the value `x` has **at that moment**.

Slide 93 — Row by row: the answer

Line	The sum	x becomes
`x += 3;`	10 + 3	13
`x -= 2;`	13 − 2	11
`x *= 4;`	11 × 4	44
`x /= 5;`	44 ÷ 5	8
`x %= 2;`	remainder of 8 ÷ 2	0

`44 / 5` is 8, not 8.8: `x` is an `int`, and `int` divided by `int` throws the decimals away (week 2).

`18_compound_assignment.c` in the Class 5 code prints `x` after every line: `13`, `11`, `44`, `8`, `0`.

Where you use these: adding up totals, changing a score, changing a bank balance.

```
int total = 100;
total -= 30;
total -= 45;
printf("Left: %d\n", total);
```

The screen shows `Left: 25`.

Slide 94 — The update does not have to be ++

The update of a loop can be any line that changes the variable. Here it is `i += 2`: add 2 after each round.

```
for (int i = 0; i <= 10; i += 2) {
    printf("%d ", i);
```

```
}
```

The screen shows `0 2 4 6 8 10`. `i` goes 0, 2, 4, 6, 8, 10, and then 12, and `12 <= 10` is false, so the loop ends.

This is `19_count_by_twos.c` in the Class 5 code. Try: count by fives from 0 to 50; print the odd numbers from 1 to 9 (start at 1).

Try it yourself. Predict the output first, then run it:

```
int x = 1;
while (x < 100) {
    printf("%d ", x);
    x *= 2;
}
```

(Answer: `1 2 4 8 16 32 64`. Each round doubles `x`. After 64 comes 128, and `128 < 100` is false, so the loop ends.)

Slide 95 — A box that collects

```
int sum = 0;

for (int i = 1; i <= 4; i++) {
    sum += i;
}

printf("%d\n", sum);
```

The screen shows `10`. The box `sum` starts at 0. In each round, `sum += i` adds the current `i` to it. This is `20_sum_1_to_4.c` in the Class 5 code. The next four slides trace it.

Slide 96 — Trace the sum: round 1

Round 1: `i` is 1. `sum` was 0. `0 + 1` is 1. `sum` is now 1.

Slide 97 — Trace the sum: round 2

Round 2: `i` is 2. `sum` was 1. `1 + 2` is 3. `sum` is now 3.

Slide 98 — Trace the sum: round 3

Round 3: `i` is 3. `sum` was 3. `3 + 3` is 6. `sum` is now 6.

Slide 99 — Trace the sum: round 4

Round 4: `i` is 4. `sum` was 6. `6 + 4` is 10. `sum` is now 10. Then `i` becomes 5, `5 <= 4` is false, the loop ends, and the `printf` shows `10`.

Round	`i`	`sum += i`	`sum` becomes
(start)			0
1	1	0 + 1	1
2	2	1 + 2	3
3	3	3 + 3	6
4	4	6 + 4	10

`1 + 2 + 3 + 4` is 10.

Slide 100 — The accumulator

A box that collects a little more in every round has a name: the **accumulator**. It is the most useful idea in all of loops, and the next class starts with it.

Try the "Try" list of `20_sum_1_to_4.c`: add up 1 to 10 (the answer is 55), then 1 to 100 (5050).

Slide 101 — Today, in six sentences

1. A loop has four parts: **start, check, body, update**.
2. `while` puts them in three places.
3. `for` puts three of them on one line.
4. No update: the loop never stops. **Ctrl+C**.
5. `i++` adds one. `sum += i` adds `i`.
6. Do not guess the output. **Trace it.**

After this class you can:

- Name the four parts of a loop.
- Write a `while` loop and a `for` loop that count up or down.
- Trace a loop with a table, and say what `x` holds after the loop.
- Stop a program that never ends with **Ctrl+C**, and find the missing update.
- Read `i++`, `i--`, `b = a++`, `b = ++a`, and `+=`, `-=`, `*=`, `/=`, `%=`.

Slide 102 — Before Friday

1. **Do Knowledge check 5** on Blackboard, in the Week 3 module: this class's slides. Six questions, three attempts, best kept, worth 0%.
2. **Vote** on Blackboard about the number of quizzes. The vote closes on Thursday at 23:59. The announcement explains it.
3. **Type `count.c` and `for.c` again, without looking** (slides 29 and 66).
4. Make each of them count from 10 down to 1.
5. **Download Class 5 code** from Blackboard: Week 3 module, item **Class 5 code**. Unzip it, open the folder in VS Code, and work through the files in order. Each file has a "Try" list at the top. Three of the programs never stop by themselves (`07`, `08` and `15`); the comment at the top says so. Ctrl+C stops them.

Next class: what loops are for. Totals, counters, and asking again until the input is right.

Slide 103 — Exit ticket

Name the four parts of a loop. (Answer: start, check, body, update. If you could not say them at once, read slide 23 again.)